Lab Guide

ROS 2 For QCar 2

Description

This document focuses on the series of launch files and nodes currently designed for QCar 2. Please review the User Manual – Connectivity for information on how to copy files onto the QCar 2.

Getting Started:

All the examples provided for QCar 2 are compatible with **ROS 2 Humble**. To get started with ROS 2 please check the following link: https://docs.ros.org/en/humble/index.html .
This guide assumes you have a basic understanding of the ROS 2 ecosystem and know how folder structures, packages and nodes work in the context of ROS.

Files Provided:

QCar 2 ROS 2 nodes can be found as part of the available research examples provided by Quanser. Using file explorer, please navigate to the following resource directory: C:/ User/<user>/Documents/Quanser/5_research/sdcs\qcar2\hardware\ros2.
Packages available for QCar 2:

- qcar2_interface
- qcar2_nodes
- qcar2_autonomy

qcar2_interfaces is a custom package for qcar2 specific elements. Custom message types are:

- **BooleanLeds.msg**
- **MotorCommands.msg**

qcar2_nodes is a Quanser design QCar2 package which configures the following information:

Sensor Centric Nodes

- **qcar2_hardware** (this node includes motor commands, LED values and publishes IMU data, battery level, joint states)
- **lidar** (publishes LiDAR info for the RP Lidar A2M12)
- **csi** (publishes image for 1 csi camera, camera ID can be modified to request information from a different CSI camera)
- **rgbd** (publishes RGB and depth information for D435 Intel RealSense camera)

User Centric Nodes

- **Command** (This node allows a user to send manual commands to the QCar using a Logitech F710 joystick)

Auxiliary Nodes

- **fixed_lidar_frame_virtual** (Transforms and rotates the LiDAR frame to align correctly with the front of the virtual QCar2).
- **fixed_lidar_frame** (Transforms and rotates the LiDAR frame to align correctly with the front of the QCar).
- **nav2_qcar_command_convert** (Converts velocity messages using a Twist message type to the correct motor command topic expected by the qcar2_hardware node)

Launch folder contains different applications to help you get started with the QCar 2.
Virtual QCar2 launch files:

- **qcar2_virtual_launch.py** (Launch file designed to publish all the sensor centric nodes for a virtual QCar2)
- **qcar2_cartographer_virtual_launch.py** (Launch file for mapping and virtual QCar2 without a command node)
- **qcar2_slam_and_nav_bringup_virtual_launch.py** (Launch file which uses Nav2 to generate commands for a virtual QCar 2 to navigate in a desired space based on a goal pose given via Rviz2

Physical QCar2 launch files

- **qcar2_launch.py** (Launch file designed to publish all the sensor centric nodes)
- **qcar2_manual_drive_launch.py** (Launch file includes sensor centric nodes and command node to manually drive the QCar 2).
- **qcar2_cartographer_launch.py** (Launch file for mapping and QCar without a command node)
- **qcar2_manual_cartographer_launch.py** (Launch file manually driving a QCar 2 and generating a map of the environment)
- **qcar2_slam_and_nav_bringup_launch.py** (Launch file which uses Nav2 to generate commands for a QCar 2 to navigate in a desired space based on a goal pose given via Rviz2)

qcar2_autonomy is a Quanser designed QCar2 package to show elements involved in Self-Driving applications.

Image processing nodes:

- **traffic_systrem_detector.py**
  This node uses an RGB image topic to perform color thresholding and contour detection to identify if there is a yield or stop sign detected.
  RGB Image information should be published using the topic name `/camera/color_image` via an image topic type.

  A `motion enable` topic is published to let the QCar know if a stop sign or yield sign has been detected and motion needs to be paused.

- **yolo_detector.py**
  This node requires `QCar2DepthAlign.rt-linux-qcar2` to read information from the RealSense D435 and provide an RGB and Depth aligned image.

  Note: This implies the QCar 2 RGBD node cannot be ran when trying to run the yolo_detector node.

A timer reads the image stream, performs preprocessing on the RGB image to be supported by YOLO v8 and then perform object detection and produce the distance to the detected object. Image information is still published under the following topics:

- `/qcar_camera/rgb`
- `/qcar_camera/depth`

A `motion enable` topic is published to let the QCar know if a stop sign or yield sign has been detected and motion needs to be paused.

Path planning, navigation, state estimation, safety:

- nav_to_pose.py

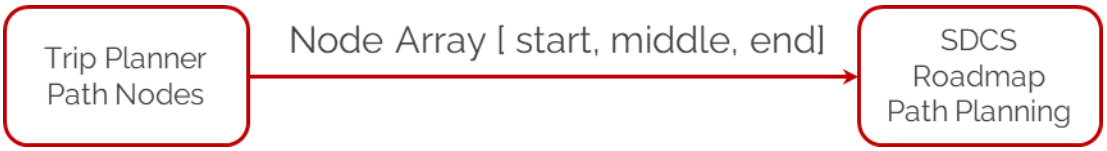
Nav to pose combines a sequence of algorithms to generate the motion commands required by the QCar 2.

Section 1: Position Estimation

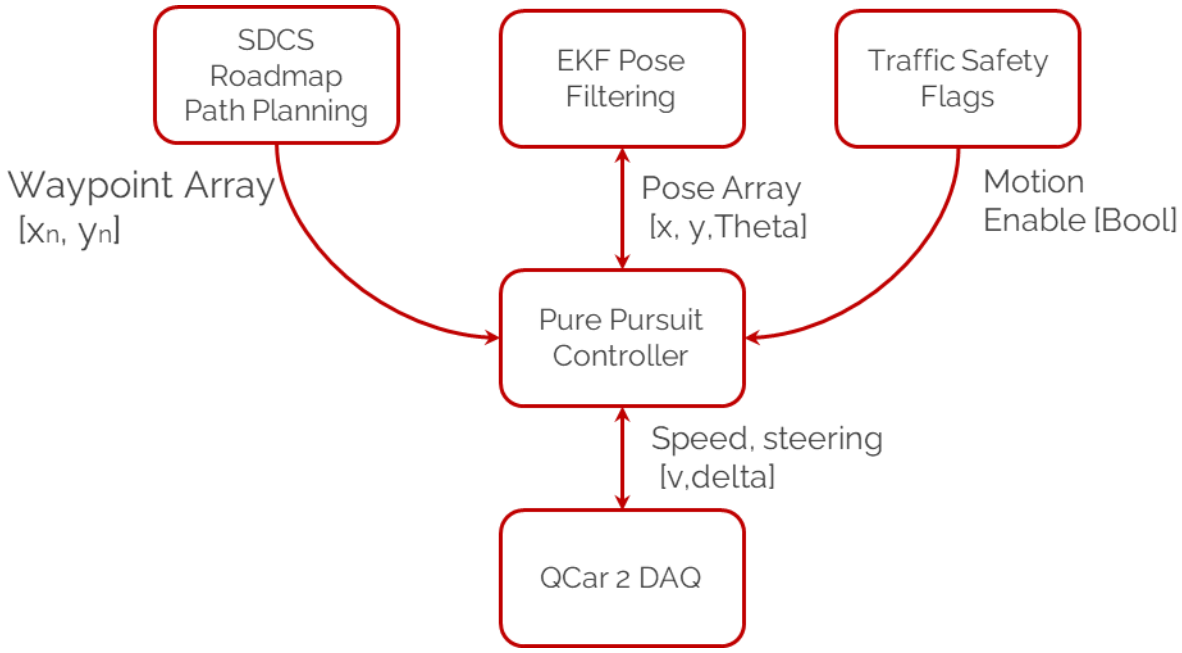
ROS Cartographer publishes a map transform to the center of the QCar2. This transformation matrix is decomposed to the X,Y,Theta elements and used with the EKF based position estimation algorithm used in the state estimation skills activity. This system is running continuously if the node is running.

Section 2: Path Planning

Parameter `node_values` defines the start and end of the local path which should be taken by the QCar2. This value is defined by the trip planner node or can be defined manually if the nav_to_pose node is tested manually. The SDCS roadmap class uses the starting and ending nodes to generate the physical waypoints used by the QCar 2 to navigate through the map. Path planning only runs when a new path sequence is given to the QCar2 or the current path is completed.



Section 3: Trajectory Tracking



Trajectory tracking involves taking the completed waypoint list provided by the SDCS Roadmap class and using the pure pursuit algorithm, complete the desired trajectory safely while stopping based on the motion enable flag from the yolo_detector (or traffic_system_detector node). EKF pose filtering provides the vehicle location used by the pure pursuit algorithm to generate the desired steering command. This node will traverse a valid waypoint trajectory provided by the SDCS roadmap class.

Route planning and behavior monitoring:

- trip_planner.py

To complete a simulated driving task, we created a trip_planner node to control the high-level behaviour of the vehicle assuming multiple stop locations are required.

Behaviour planner has the following super states:

- Moving to taxi/starting zone
- Performing trip

Moving to taxi/starting zone

This super state makes sure the QCar2 can move from the origin calibration spot to the node designed to be the starting taxi location. This state only happens once when the node has started. If the node is stopped and started you will need to manually place the QCar 2 at the origin calibration location and restart the cartography mapping package.

Performing trip

This super state handles the trip array to make sure the nav_to_pose.py node generates a path via the correct starting and ending nodes. The trip planner makes sure the QCar 2 is back to the starting zone prior to accepting a new trip plan.

Running Examples

Prior to running any ROS 2 example, please make sure you have sourced **humble** as the desired ROS 2 distribution in your current terminal session.

```
source /opt/ros/humble/setup.bash
```

Navigate to the **ros2/src** folder on the QCar 2 root directory. Copy **qcar2_interfaces** and **qcar2_nodes** inside this directory.

Note: If this is the first time compiling the ros2 workspace navigate to the **/ros2** folder and use the command **colcon build** to compile the ros2 workspace. Once the workspace has compiled successfully it needs to be sourced as a ROS 2 package using the following command:

```
source install/setup.bash
```

Running nodes:

To run any ROS 2 node, use the following command:

```
ros2 run <package name> <node name(s)>
```

As an example, to run the QCar2 LiDAR node, use the following command:

```
ros2 run qcar2_nodes lidar
```

Note: Nodes run 1 per terminal session. If you want to run multiple nodes either use multiple terminal sessions or combine them into launch files.

Running launch files:

Launch files combine a series of nodes into a unique application. To run any launch node use the following command:

```
ros2 launch <package name> <name of launch file.py>
```

As an example, if you want to run the QCar 2 manual drive example use the following command:

```
ros2 launch qcar2_nodes qcar2_manual_drive_launch.py
```